

Count Beautiful Splits in an Array

Company: Amazon

Round: Online Assessment (OA)

Year: 2025

Difficulty: Medium

Topic: Arrays, Prefix Matching, String/Sequence Matching, LCP

Source: LeetCode 3388 - Count Beautiful Splits in an Array

Problem Description

You are given an array `nums`.

A split of an array `nums` is **beautiful** if:

- The array `nums` is split into three non-empty subarrays: `nums1`, `nums2`, and `nums3`.
- `nums` can be formed by concatenating `nums1`, `nums2`, and `nums3` in that order.
- The subarray `nums1` is a **prefix of `nums2`**, **OR** `nums2` is a ****prefix of `nums3`**.

Return the number of ways you can make this split.

Input Format

The first line contains an integer `N` — the length of the array.

The second line contains `N` space-separated integers representing `nums`.

Output Format

Return a single integer representing the number of **beautiful splits** of the array.

Constraints

- $1 \leq \text{nums.length} \leq 5000$
- $0 \leq \text{nums}[i] \leq 50$
- The array must be divided into **three non-empty subarrays**.
- The relative order of elements must remain unchanged..

Sample Input / Output

Example 1

Input:

`nums = [1,1,2,1]`

Output:

2

Explanation:

The beautiful splits are:

Split 1:

```
nums1 = [1]
nums2 = [1, 2]
nums3 = [1]
```

Here, `nums1 = [1]` is a prefix of `nums2 = [1, 2]`.

Split 2:

```
nums1 = [1]
nums2 = [1]
nums3 = [2, 1]
```

Here, `nums1 = [1]` is a prefix of `nums2 = [1]`.

Therefore, the answer is:

2

Example 2

Input:

```
nums = [1, 2, 3, 4]
```

Output:

0

Explanation:

There is no way to split the array into three non-empty subarrays such that either `nums1` is a prefix of `nums2` or `nums2` is a prefix of `nums3`.

Therefore, the answer is 0.

Input:

```
houses = [1, 2, 3]
heaters = [2]
```

Output:

1

Explanation:

The only heater was placed at position 2. With radius 1, the heater covers houses at positions 1, 2, and 3.

Approach to Solve This Problem:

We need to choose two split positions.

Suppose:

```
nums = [ ... nums1 | ... nums2 | ... nums3 ]
```

Let:

- i = end position of `nums1`
- j = end position of `nums2`

Then:

```
nums1 = nums[0 ... i-1]  
nums2 = nums[i ... j-1]  
nums3 = nums[j ... n-1]
```

Both `nums1` and `nums2` must be non-empty, and `nums3` must also be non-empty.

The condition is:

```
nums1 is prefix of nums2  
OR  
nums2 is prefix of nums3
```

1. Check whether `nums1` is a prefix of `nums2`

Suppose:

```
len1 = i  
len2 = j - i
```

For `nums1` to be a prefix of `nums2`, we need:

```
len1 <= len2
```

and:

```
nums[k] == nums[i + k]
```

for every:

```
0 <= k < len1
```

So we compare the first `len1` elements of `nums2` with `nums1`.

2. Check whether `nums2` is a prefix of `nums3`

Similarly, `nums2` must not be longer than `nums3`.

We need:

```
len2 <= len3
```

where:

```
len3 = n - j
```

Then compare:

```
nums[i + k] == nums[j + k]
```

for:

```
0 <= k < len2
```

Optimized Approach Using LCP

A direct comparison for every pair of split positions can take $O(n^3)$ in the worst case.

We can optimize the prefix comparisons using an **LCP (Longest Common Prefix)** table.

Define:

```
lcp[i][j]
```

as the length of the longest common prefix between:

```
nums[i...]
```

and:

```
nums[j...]
```

We can calculate this from right to left.

If:

```
nums[i] == nums[j]
```

then:

```
lcp[i][j] = 1 + lcp[i + 1][j + 1]
```

Otherwise:

```
lcp[i][j] = 0
```

Now prefix checking becomes $O(1)$.

For example, `nums1` is a prefix of `nums2` if:

```
lcp[0][i] >= len(nums1)
```

And `nums2` is a prefix of `nums3` if:

```
lcp[i][j] >= len(nums2)
```

This gives an overall $O(n^2)$ solution.

Java Solution:

```
class Solution {
```

```
    public int beautifulSplits(int[] nums) {
```

```
        int n = nums.length;
```

```
        // lcp[i][j] = longest common prefix length
```

```
        // between nums[i...] and nums[j...]
```

```
        int[][] lcp = new int[n + 1][n + 1];
```

```
        for (int i = n - 1; i >= 0; i--) {
```

```
            for (int j = n - 1; j >= 0; j--) {
```

```
                if (nums[i] == nums[j]) {
```

```
                    lcp[i][j] = 1 + lcp[i + 1][j + 1];
```

```
                }
```

```
            }
```

```
        }
```

```
        int answer = 0;
```

```
        // i = end of nums1
```

```
        // j = end of nums2
```

```
        for (int i = 1; i < n - 1; i++) {
```

```
            for (int j = i + 1; j < n; j++) {
```

```
                int len1 = i;
```

```
                int len2 = j - i;
```

```
                int len3 = n - j;
```

```
                boolean firstCondition = false;
```

```
                boolean secondCondition = false;
```

```
                // nums1 is a prefix of nums2
```

```
                if (len1 <= len2 &&
```

```
                    lcp[0][i] >= len1) {
```

```
                    firstCondition = true;
```

```
                }
```

```
// nums2 is a prefix of nums3
if (len2 <= len3 &&
    lcp[i][j] >= len2) {

    secondCondition = true;
}

if (firstCondition || secondCondition) {
    answer++;
}
}
}

return answer;
}
}
```

Space Complexity: $O(n^2)$

The LCP table requires:

$$(n + 1) \times (n + 1)$$

integer entries.